PYTHON PROGRAMMING ACTIVITY-BASED LEARNING (ABL)

Complete Report — 10 Selected Case Studies

Institution	Dr. RVR NRI Institute of Technology
Programme	B. Tech – CSM (AI & ML)
Student Name	SINDHURA A
Roll Number	26001056
Branch / Section	CSM / 4
Course	Python Programming

This report contains the required documentation for each selected case study, together with the output screenshots taken after running the corresponding Python file.

Report Guide

Mandatory sections included for every case: Problem Statement, Objectives, Assumptions, Algorithm, Flowchart, Pseudocode, Python Source Code, Test Cases, Output Screenshot(s), and Conclusion.

Selected case studies

Case	Case Study	Python file
2	Progressive Electricity Billing	02_Progressive Electricity Billing.py
13	ATM Transaction Sequence Simulation	13_ATM Transaction Sequence Simulation.py
29	Password Policy Validation Engine	29_Password Policy Validation Engine.py
42	Student Marks Matrix Analysis	42_Student Marks Matrix Analysis.py
55	Tuple-Based Transaction Audit	55_Tuple-Based Transaction Audit.py
60	Manufacturing Quality Control Matrix	60_Manufacturing Quality Control Matrix.py
66	Student Academic Risk Classification	66_Student Academic Risk Classification.py
77	Smart Electricity Billing for Multiple Apartments	77_Smart Electricity Billing for Multiple Apartments.py
85	Smart Classroom Timetable Allocation	85_Smart Classroom Timetable Allocation.py
98	University Academic Progression Audit	98_University Academic Progression Audit.py

Case 2: Progressive Electricity Billing

Python file: 02_Progressive Electricity Billing.py

1. Problem Statement

Calculate the electricity bill of a household from the number of units consumed, using slab-wise (progressive) rates.

2. Objectives

- Read the units consumed from the user.
- Apply different rates to different consumption slabs using if-elif-else.
- Display the final electricity bill.

3. Assumptions

- Slab 1: first 100 units at Rs 2 per unit.
- Slab 2: next 100 units (101 to 200) at Rs 3 per unit.
- Slab 3: units above 200 at Rs 5 per unit.
- Units are entered as a whole number.

4. Algorithm

1. Start.
2. Read the units consumed.
3. If units ≤ 100, bill = units × 2.
4. Else if units ≤ 200, bill = 100 × 2 + (units − 100) × 3.
5. Else bill = 200 + 300 + (units − 200) × 5.
6. Print the bill.
7. Stop.

5. Flowchart

START → Read units → units ≤ 100? → Yes: bill = units × 2 → No: units ≤ 200? → Yes: bill = 200 + (units − 100) × 3 → No: bill = 500 + (units − 200) × 5 → Print bill → STOP

6. Pseudocode

1. Read units.
2. If units ≤ 100: bill = units × 2.
3. Else if units ≤ 200: bill = 100 × 2 + (units − 100) × 3.
4. Else: bill = 200 + 300 + (units − 200) × 5.
5. Print bill.

7. Python Source Code

```
units = int(input("Enter units: "))
if units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
else:
    bill = 200 + 300 + (units - 200) * 5
print("Electricity Bill:", bill)
```

8. Test Cases

- Test 1: 50 units → Electricity Bill: 100
- Test 2: 150 units → Electricity Bill: 350
- Test 3: 250 units → Electricity Bill: 750

9. Output Screenshot(s)

```
PS C:\Users\HP\OneDrive\Desktop\pythonactivity> & C:\Users\HP\AppData\Local\Programs\Python\Py
thon314\python.exe "c:/Users/HP/OneDrive/Desktop/pythonactivity/programs/02_Progressive Electr
icity Billing.py"
Enter units: 50
Electricity Bill: 100
PS C:\Users\HP\OneDrive\Desktop\pythonactivity>
PS C:\Users\HP\OneDrive\Desktop\pythonactivity> & C:\Users\HP\AppData\Local\Programs\Python\Py
thon314\python.exe "c:/Users/HP/OneDrive/Desktop/pythonactivity/programs/02_Progressive Electr
icity Billing.py"
Enter units: 150
Electricity Bill: 350
PS C:\Users\HP\OneDrive\Desktop\pythonactivity> & C:\Users\HP\AppData\Local\Programs\Python\Py
thon314\python.exe "c:/Users/HP/OneDrive/Desktop/pythonactivity/programs/02_Progressive Electr
icity Billing.py"
Enter units:
```

10. Conclusion

The program calculated the electricity bill correctly using slab rates. It shows how if-elif-else statements are used for slab-based calculation.

Case 13: ATM Transaction Sequence Simulation

Python file: 13_ATM Transaction Sequence Simulation.py

1. Problem Statement

Simulate a sequence of three ATM withdrawals from an account, accept only valid transactions, and show the successful withdrawals and the remaining balance.

2. Objectives

- Use a loop to process a fixed number of transactions.
- Validate each withdrawal amount using conditions.
- Store successful withdrawals in a list.
- Display the list and the remaining balance.

3. Assumptions

- The opening balance is Rs 5000.
- Exactly 3 withdrawal attempts are processed.
- A withdrawal is valid only if the amount is greater than 0 and not more than the current balance.
- Invalid transactions are skipped and do not change the balance.

4. Algorithm

1. Start.
2. Set balance = 5000 and create an empty list transactions.
3. Repeat 3 times: read the withdrawal amount.
4. If 0 < amount ≤ balance, subtract it from the balance, add it to the list and print 'Withdrawal successful'.
5. Otherwise print 'Invalid transaction'.
6. Print the list of transactions and the remaining balance.
7. Stop.

5. Flowchart

START → balance = 5000, transactions = [ ] → Repeat 3 times: read amount → Valid (0 < amount ≤ balance)? → Yes: update balance and list → No: print Invalid → End of loop → Print transactions and balance → STOP

6. Pseudocode

1. balance = 5000; transactions = [ ].
2. For i = 1 to 3: read amount.
3. If amount > 0 and amount ≤ balance: balance = balance − amount; append amount; print 'Withdrawal successful'.
4. Else: print 'Invalid transaction'.
5. Print transactions and balance.

7. Python Source Code

```
balance = 5000
transactions = []
for i in range(3):
    amount = int(input("Enter withdrawal amount: "))
    if amount > 0 and amount <= balance:
        balance -= amount
        transactions.append(amount)
        print("Withdrawal successful")
    else:
        print("Invalid transaction")
print("Transactions:", transactions)
print("Remaining Balance:", balance)
```

8. Test Cases

- Test 1: 500, 1000, 500 → Transactions [500, 1000, 500]; Remaining Balance: 3000
- Test 2: 2000, 1000, 500 → Transactions [2000, 1000, 500]; Remaining Balance: 1500
- Test 3: 6000, 500, 500 → 6000 is Invalid transaction; Transactions [500, 500]; Remaining Balance: 4000

9. Output Screenshot(s)



10. Conclusion

The program simulated ATM withdrawals and rejected an amount larger than the balance. It shows the use of loops, lists and conditions together.

Case 29: Password Policy Validation Engine

Python file: 29_Password Policy Validation Engine.py

1. Problem Statement

Check whether a password satisfies the password policy and display whether it is valid or invalid.

2. Objectives

- Read a password as a string.
- Check its length and character types using string methods.
- Combine the rules using logical operators.

3. Assumptions

- The password must have at least 8 characters.
- It must contain at least one uppercase letter, one lowercase letter and one digit.
- A password that breaks any rule is invalid.

4. Algorithm

1. Start.
2. Read the password.
3. Check that the length is at least 8.
4. Check that at least one character is uppercase, one is lowercase and one is a digit.
5. If all conditions are true, print 'Valid Password'.
6. Otherwise print 'Invalid Password'.
7. Stop.

5. Flowchart

START → Read password → Length ≥ 8 and has uppercase and has lowercase and has digit? → Yes: print Valid Password → No: print Invalid Password → STOP

6. Pseudocode

1. Read password.
2. If length(password) ≥ 8 and any character isupper and any character islower and any character isdigit:
3. print 'Valid Password'
4. Else: print 'Invalid Password'.

7. Python Source Code

```
password = input("Enter password: ")
if (len(password) >= 8 and
    any(c.isupper() for c in password) and
    any(c.islower() for c in password) and
    any(c.isdigit() for c in password)):
    print("Valid Password")
else:
    print("Invalid Password")
```

8. Test Cases

- Test 1: Hello123 → Valid Password
- Test 2: hello123 → Invalid Password (no uppercase letter)
- Test 3: ABC123 → Invalid Password (shorter than 8 and no lowercase letter)

9. Output Screenshot(s)



10. Conclusion

The program validated passwords against the policy rules. It shows string handling, the any() function and logical operators.

Case 42: Student Marks Matrix Analysis

Python file: 42_Student Marks Matrix Analysis.py

1. Problem Statement

Store the marks of several students in several subjects as a matrix (nested list) and find the total of each student and the average of each subject.

2. Objectives

- Create and use a nested list of marks.
- Find the total of each student (row-wise).
- Find the average of each subject (column-wise).

3. Assumptions

- There are 3 students and 3 subjects.
- The marks are stored in the program as a 3 × 3 matrix.
- Rows represent students and columns represent subjects.

4. Algorithm

1. Start.
2. Create the 3 × 3 marks matrix.
3. For each row, print the student number and the sum of the row.
4. For each column j, add the marks of all students in that column.
5. Divide the column total by 3 and print the subject average.
6. Stop.

5. Flowchart

START → Create marks matrix → For each student: print row total → For each subject: add column marks → Subject average = total / 3 → Print average → STOP

6. Pseudocode

1. marks = 3 × 3 matrix.
2. For each row i: print i + 1 and sum(row).
3. For each column j: total = sum of marks[i][j] over all rows.
4. Print total / 3 as the subject average.

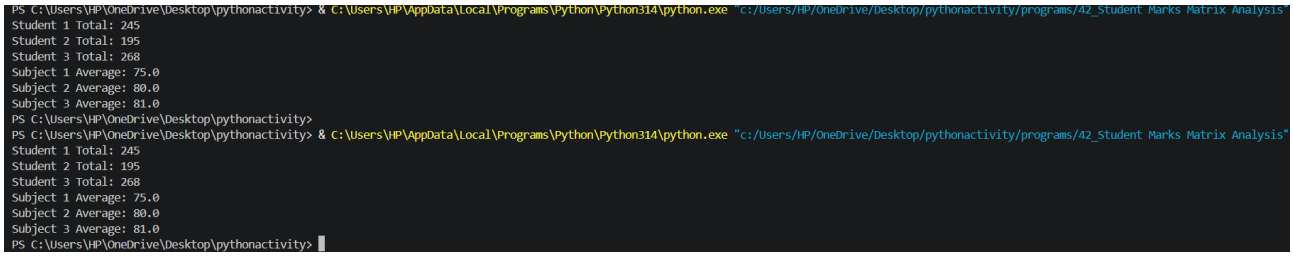
7. Python Source Code

```
marks = [
    [80, 75, 90],
    [60, 70, 65],
    [85, 95, 88]
]
for i, row in enumerate(marks):
    print("Student", i + 1, "Total:", sum(row))
for j in range(3):
    total = sum(marks[i][j] for i in range(3))
    print("Subject", j + 1, "Average:", total / 3)
```

8. Test Cases

- Test 1: Student 1 (80, 75, 90) → Total 245
- Test 2: Student 2 (60, 70, 65) → Total 195
- Test 3: Student 3 (85, 95, 88) → Total 268
- Subject averages → 75.0, 80.0 and 81.0

9. Output Screenshot(s)



10. Conclusion

The program analysed the marks matrix row-wise and column-wise. It shows how nested lists are traversed with loops and the sum() function.

Case 55: Tuple-Based Transaction Audit

Python file: 55_Tuple-Based Transaction Audit.py

1. Problem Statement

Store transactions in a tuple and audit them by finding any amount that appears more than once.

2. Objectives

- Store data in a tuple.
- Compare every pair of elements using nested loops.
- Display the duplicate transactions.

3. Assumptions

- Transactions are stored as a tuple of amounts, which cannot be changed after creation.
- A duplicate is an amount that occurs at least twice in the tuple.

4. Algorithm

1. Start.
2. Create the tuple of transactions and print it.
3. For each position i, compare the element with every element after it (position j > i).
4. If the two elements are equal, print 'Duplicate' and the amount.
5. Stop.

5. Flowchart

START → Create tuple → Print tuple → For each i → For each j after i → transactions[i] = transactions[j]? → Yes: print Duplicate → Next pair → STOP

6. Pseudocode

1. transactions = (1000, 2000, 1000, 3000); print it.
2. For i = 0 to length − 1:
3. For j = i + 1 to length − 1:
4. If transactions[i] = transactions[j]: print 'Duplicate', transactions[i].

7. Python Source Code

```
transactions = (1000, 2000, 1000, 3000)
print("Transactions:", transactions)
for i in range(len(transactions)):
    for j in range(i + 1, len(transactions)):
        if transactions[i] == transactions[j]:
            print("Duplicate:", transactions[i])
```

8. Test Cases

- Test 1: (1000, 2000, 1000, 3000) → Duplicate: 1000
- Test 2: (100, 200, 100) → Duplicate: 100
- Test 3: (500, 600, 700) → no Duplicate line is printed (no repeated amount)
- Test 4: (1000, 1000, 2000) → Duplicate: 1000

9. Output Screenshot(s)



10. Conclusion

The program detected repeated transactions in a tuple. It shows tuple creation, indexing and nested loops for comparing pairs.

Case 60: Manufacturing Quality Control Matrix

Python file: 60_Manufacturing Quality Control Matrix.py

1. Problem Statement

Analyse a quality-control matrix of manufactured products and find the total products, the defective products and the defect percentage.

2. Objectives

- Traverse a 2D list with nested loops.
- Count total and defective items using counters.
- Calculate the defect percentage.

3. Assumptions

- The matrix has 3 rows and 4 columns (12 products).
- 1 means a good product and 0 means a defective product.

4. Algorithm

1. Start.
2. Create the quality matrix; set total = 0 and defects = 0.
3. For each row and each item in the row, increase total by 1.
4. If the item is 0, increase defects by 1.
5. Calculate percentage = (defects / total) × 100.
6. Print total products, defective products and defect percentage.
7. Stop.

5. Flowchart

START → Create matrix, total = 0, defects = 0 → For each row → For each item → total + 1 → item = 0? → Yes: defects + 1 → End loops → percentage = defects / total × 100 → Print results → STOP

6. Pseudocode

1. total = 0; defects = 0.
2. For each row in quality: for each item in row:
3. total = total + 1
4. If item = 0: defects = defects + 1
5. percentage = defects / total × 100; print total, defects, percentage.

7. Python Source Code

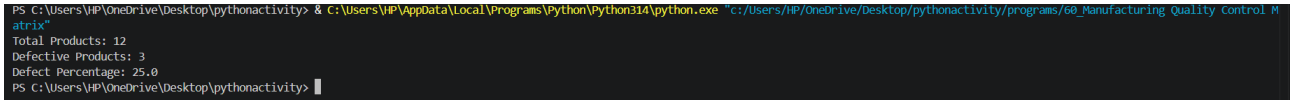
```
quality = [
    [1, 0, 1, 1],
    [1, 1, 0, 1],
    [0, 1, 1, 1]
]
total = 0
defects = 0
for row in quality:
    for item in row:
        total += 1
        if item == 0:
            defects += 1
percentage = (defects / total) * 100
print("Total Products:", total)
print("Defective Products:", defects)
print("Defect Percentage:", percentage)
```

8. Test Cases

- Test 1: 12 products, 3 defects → Defect Percentage: 25.0
- Test 2: 12 products, 4 defects → about 33.33 %

- Test 3: 12 products, 0 defects → 0.0 %

9. Output Screenshot(s)



10. Conclusion

The program counted the defective products in the matrix and calculated the defect percentage. It shows nested loops, counters and arithmetic on a 2D list.

Case 66: Student Academic Risk Classification

Python file: 66_Student Academic Risk Classification.py

1. Problem Statement

Classify students into Low, Medium and High academic risk categories based on their marks.

2. Objectives

- Process a list of marks using a loop.
- Use if-elif-else to classify each mark.
- Display the risk level of every student.

3. Assumptions

- Marks of 60 and above are Low Risk.
- Marks from 40 to 59 are Medium Risk.
- Marks below 40 are High Risk.
- The marks are stored in a list in the program.

4. Algorithm

1. Start.
2. Create the list of marks.
3. For each mark, check: if mark ≥ 60 print 'Low Risk'.
4. Else if mark ≥ 40 print 'Medium Risk'.
5. Else print 'High Risk'.
6. Stop.

5. Flowchart

START → Create marks list → For each mark → mark ≥ 60? → Yes: Low Risk → No: mark ≥ 40? → Yes: Medium Risk → No: High Risk → Next mark → STOP

6. Pseudocode

1. marks = [85, 45, 30, 70, 25].
2. For each mark in marks:
3. If mark ≥ 60: print mark, 'Low Risk'
4. Else if mark ≥ 40: print mark, 'Medium Risk'
5. Else: print mark, 'High Risk'.

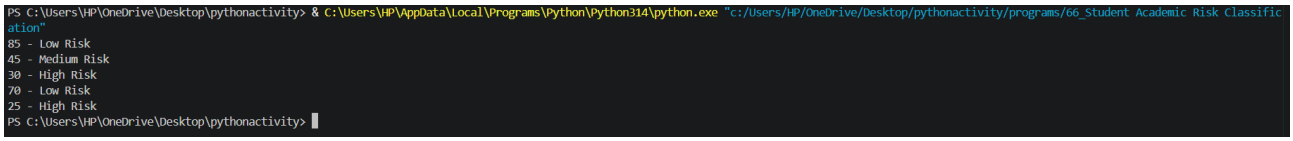
7. Python Source Code

```
marks = [85, 45, 30, 70, 25]
for mark in marks:
    if mark >= 60:
        print(mark, "- Low Risk")
    elif mark >= 40:
        print(mark, "- Medium Risk")
    else:
        print(mark, "- High Risk")
```

8. Test Cases

- Test 1: 85 → Low Risk
- Test 2: 45 → Medium Risk
- Test 3: 30 → High Risk
- Also tested: 70 → Low Risk and 25 → High Risk

9. Output Screenshot(s)



10. Conclusion

The program classified every student into the correct risk category. It shows the use of a loop with if-elif-else conditions.

Case 77: Smart Electricity Billing for Multiple Apartments

Python file: 77_Smart Electricity Billing for Multiple Apartments.py

1. Problem Statement

Calculate the electricity bills of several apartments using slab rates and identify the apartment with the highest bill.

2. Objectives

- Process a list of units using a loop.
- Apply slab rates to each apartment.
- Store the bills in a list and find the maximum using max() and index().

3. Assumptions

- Units of the apartments are stored in a list: 100, 250, 150 and 300.
- Up to 100 units cost Rs 2 per unit; the next 100 units cost Rs 3; units above 200 cost Rs 5.
- Apartment numbers start from 1.

4. Algorithm

1. Start.
2. Create the list of units and an empty list bills.
3. For each value u, find the bill using the slab rates.
4. Append the bill to the list.
5. Print the list of bills.
6. Find the highest bill using max() and its apartment number using index() + 1.
7. Stop.

5. Flowchart

START → units list, bills = [ ] → For each u → Slab check (≤ 100, ≤ 200, above) → Calculate bill → Append to bills → End loop → Highest bill and its apartment → Print → STOP

6. Pseudocode

1. units = [100, 250, 150, 300]; bills = [ ].
2. For each u in units:
3. If u ≤ 100: bill = u × 2
4. Else if u ≤ 200: bill = 200 + (u − 100) × 3
5. Else: bill = 500 + (u − 200) × 5
6. Append bill to bills.
7. Print bills, max(bills) and index of max(bills) + 1.

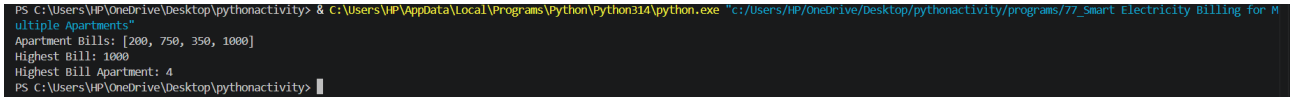
7. Python Source Code

```
units = [100, 250, 150, 300]
bills = []
for u in units:
    if u <= 100:
        bill = u * 2
    elif u <= 200:
        bill = 200 + (u - 100) * 3
    else:
        bill = 500 + (u - 200) * 5
    bills.append(bill)
print("Apartment Bills:", bills)
print("Highest Bill:", max(bills))
print("Highest Bill Apartment:", bills.index(max(bills)) + 1)
```

8. Test Cases

- Test 1: Apartment 1, 100 units → Bill 200
- Test 2: Apartment 2, 250 units → Bill 750
- Test 3: Apartment 3, 150 units → Bill 350
- Test 4: Apartment 4, 300 units → Bill 1000 (highest; Apartment 4)

9. Output Screenshot(s)



10. Conclusion

The program calculated the bills of all apartments and found the highest one. It shows lists, loops, slab calculation and the max() and index() functions.

Case 85: Smart Classroom Timetable Allocation

Python file: 85_Smart Classroom Timetable Allocation.py

1. Problem Statement

Check a classroom timetable and detect clashes, where two subjects are scheduled on the same day at the same time.

2. Objectives

- Store timetable entries as tuples in a list.
- Compare every pair of entries using nested loops.
- Use tuple slicing to compare the day and time.

3. Assumptions

- Each entry is a tuple of (day, time, subject).
- Two entries clash if their day and time are both the same.
- The timetable is stored in the program.

4. Algorithm

1. Start.
2. Create the timetable list of tuples.
3. For each entry i, compare it with every later entry j.
4. If the first two items (day and time) are equal, print 'Clash' with both entries.
5. After all pairs are checked, print 'Timetable Checked'.
6. Stop.

5. Flowchart

START → Create timetable → For each i → For each j after i → Day and time equal? → Yes: print Clash → No: next pair → End loops → Print Timetable Checked → STOP

6. Pseudocode

1. timetable = list of (day, time, subject) tuples.
2. For i = 0 to length − 1:
3. For j = i + 1 to length − 1:
4. If timetable[i][0:2] = timetable[j][0:2]: print 'Clash', timetable[i], timetable[j]
5. Print 'Timetable Checked'.

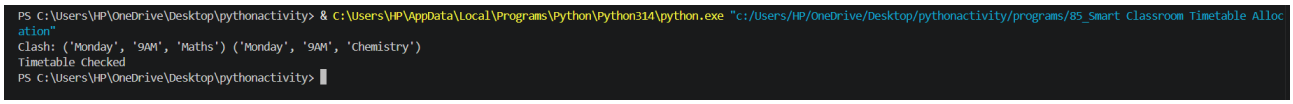
7. Python Source Code

```
timetable = [
    ("Monday", "9AM", "Maths"),
    ("Monday", "10AM", "Physics"),
    ("Tuesday", "9AM", "English"),
    ("Monday", "9AM", "Chemistry")
]
for i in range(len(timetable)):
    for j in range(i + 1, len(timetable)):
        if timetable[i][0:2] == timetable[j][0:2]:
            print("Clash:", timetable[i], timetable[j])
print("Timetable Checked")
```

8. Test Cases

- Test 1: Same day and time (Monday 9AM: Maths and Chemistry) → Clash detected
- Test 2: Different day → no clash (only 'Timetable Checked' is printed)
- Test 3: Different time → no clash (only 'Timetable Checked' is printed)

9. Output Screenshot(s)



10. Conclusion

The program found the clash between Maths and Chemistry on Monday at 9AM. It shows tuples, slicing and nested loops for comparing pairs.

Case 98: University Academic Progression Audit

Python file: 98_University Academic Progression Audit.py

1. Problem Statement

Decide whether students are eligible to progress to the next level based on their marks, attendance and prerequisite completion.

2. Objectives

- Apply multiple conditions together using logical operators.
- Process a list of marks using a loop.
- Display the eligibility of every student.

3. Assumptions

- A student is eligible only if marks ≥ 40, attendance ≥ 75 and the prerequisite is completed.
- Attendance is 80 and the prerequisite is True for the sample data.
- Marks are stored in a list: 75, 65 and 35.

4. Algorithm

1. Start.
2. Create the marks list and set attendance = 80 and prerequisite = True.
3. For each mark, check mark ≥ 40 and attendance ≥ 75 and prerequisite.
4. If all are true print 'Eligible'.
5. Otherwise print 'Not Eligible'.
6. Stop.

5. Flowchart

START → marks list, attendance, prerequisite → For each mark → mark ≥ 40 and attendance ≥ 75 and prerequisite? → Yes: Eligible → No: Not Eligible → Next mark → STOP

6. Pseudocode

1. marks = [75, 65, 35]; attendance = 80; prerequisite = True.
2. For each mark in marks:
3. If mark ≥ 40 and attendance ≥ 75 and prerequisite: print mark, 'Eligible'
4. Else: print mark, 'Not Eligible'.

7. Python Source Code

```
marks = [75, 65, 35]
attendance = 80
prerequisite = True
for mark in marks:
    if mark >= 40 and attendance >= 75 and prerequisite:
        print(mark, "- Eligible")
    else:
        print(mark, "- Not Eligible")
```

8. Test Cases

- Test 1: Marks 75, attendance 80, prerequisite True → Eligible
- Test 2: Marks 35, attendance 80, prerequisite True → Not Eligible
- Test 3: Marks 65, attendance 60, prerequisite True → Not Eligible (attendance below 75)
- Test 4: Marks 75, attendance 80, prerequisite False → Not Eligible

9. Output Screenshot(s)



10. Conclusion

The program checked all the eligibility conditions and displayed the result for each student. It shows how logical operators combine several conditions.